

Wahla - Logic

Wahla is a system that understands you and your spending habits, then it makes a digital twin of you.

The digital twin will simulate your habits and decisions to give you a picture of how your financial situation would be based on your lifestyle.

Knowing you will enable the system to help you make better decisions and build more efficient habits.

How would it do that?

Using the 50 30 20 Rule

divide your income to:

- 50% needs
- 30% wants
- 20% savings

When you want to do a financial action it will consider if it fits the 50 30 20 rule or not

Ideal goal is the divide rule but it not always the situation so.

Base Logic

When the user makes a spending **decision**, classify it first:

1. **Is it a need or a want?**
 - If it's a **need** → **Add** it.
 - If it's a **want** → continue.
2. **Have “wants” already exceeded 30%?**
 - If **No** → **Add** it.
 - If **Yes** → continue.
3. **Can it be paid in installments?**
 - If **No** → **Don't** do it.
 - If **Yes** → continue.
4. **Will “needs” exceed 50% if you add it?**
 - If **No** → **Add** it.
 - If **Yes** → **Don't** do it.

Monthly adjustment rules (to move back toward 50/30/20)

1. **If wants > 30% and savings < 20%**
 - Decrease **wants** by 2–3% each month (and shift that amount to **savings**) until you reach **30% wants** and **20% savings**.
2. **If needs are 20–30% and wants > 50%**
 - Increase **savings** by 2–3% each month (taking it mainly from **wants**) until savings reaches **25–30%** per month.
3. **If needs > 50%**
 - Suggest reducing **wants** (temporarily) and keep **needs** at **20%**.

Passive product suggestion layer

This sits **on top of** the existing 50/30/20 decision engine. It never blocks or executes a transaction — it only surfaces a matching bank product when the twin detects an imbalance that a product could actually fix. The user always makes the final call.

Trigger rule (applies to everything below): only surface a suggestion if:

1. A qualifying condition exists — either:
 - a) a **ratio imbalance** (wants > 30%, savings < 20%, or needs > 50%), or
 - b) a **standalone cost inefficiency**, independent of the ratios (idle cash earning 0%, a card tier that doesn't match usage, or a business cash-flow gap for flagged business users). Sections 1, 4, and 5 fall under this category and do not require a ratio imbalance to fire — fixing an inefficiency is worth doing even when the budget is already balanced.
2. A product from the catalog measurably improves that specific condition, AND
3. The same suggestion hasn't been shown in the last 30 days (avoid nagging).
4. **Idle cash → savings account tier upgrade**
 - **Signal:** savings % is fine/high, but money is sitting in a current/checking account earning ~0%, OR the user is already in a savings product but under-tiered for their balance.
 - **Logic:**
 - **buffer = 1 month of needs_spending # same floor used by the nudge below**
 - IF **avg_monthly_current_account_balance > buffer** and the user is **not** in a savings product:
 - → Suggest opening a savings account (Daily Balance or Minimum Balance type)
 - (only the amount above the buffer is what gets suggested for the sweep — never the full balance)
 - If the user is in a savings product:
 - Pick the best tier based on average monthly balance (example tiers):

- $\leq 1,000,000 \rightarrow 1.50\% - 2.00\%$ AER
 - $\leq 3,000,000 \rightarrow 1.75\% - 2.22\%$ AER
 - $> 3,000,000 \rightarrow 2.20\% - 3.00\%$ AER
 - If `current_tier < best_tier` $\rightarrow$ suggest upgrading/moving balance to reach `best_tier`.
- **Extra nudge tied to 50/30/20:** if **savings < 20%** but the user has idle current-account cash above a small buffer (e.g., **1 month of needs**), suggest auto-sweeping the idle cash into the Daily Balance Savings account first (a “free win” before asking them to cut spending).
- 5. **Needs > 50% because of a big “need” purchase (car/home) $\rightarrow$ right-size the financing**
 - **Precedence vs. the base logic's "needs > 50%" monthly adjustment:** this section is *preventive* — it runs at decision-time, before a new need purchase is committed to. The base logic's monthly rule (temporarily cut savings to 10–15%, keep wants at 20–30%) is *reactive* — it applies only to needs spending that's already committed and can't be renegotiated at decision-time (existing rent, an existing loan already signed, etc.). If this section finds no financing tenor fits, the answer is "don't do it / scale down the purchase," not "cut savings to force it to fit" — the reactive rule never overrides a "don't do it" from this section.
 - **Signal:** the user is about to make, or already carries, a large *need* installment (car/property) that pushes needs past 50%.
 - **Core calculation:**
 - `remaining_needs_headroom = (50% * income) - current_needs_spending`
 - **Car financing logic:**
 - For each available tier/tenor (e.g., 4yr, 5yr @3.00%, 5yr @3.61%, ...):
 - If `monthly_installment(tier) <= remaining_needs_headroom` $\rightarrow$ mark it as a candidate.
 - IF `candidate_tiers` not empty:
 - $\rightarrow$ Suggest the SHORTEST tenor among candidates (lowest total profit paid — consistent with the real-estate rule below and with "never increase total cost unless it's the only way to fit")
 - $\rightarrow$ Only surface a longer tenor as a secondary/fallback option, and only if the shortest-fitting tenor leaves less than ~2% headroom below the 50% line (i.e. flag it as "tight" and offer the longer term as a safety margin, clearly labeled as costing more overall)
 - If no candidates $\rightarrow$ **do not** suggest financing; instead flag that the purchase doesn't fit even at the longest available term (needs will exceed 50% regardless).

- **Real-estate financing logic:**
 - For each term (10/15/20 years):
 - `installment = compute_installment(property_value, financing_ratio ≤ 90%, term_profit_rate)`
 - If `installment ≤ remaining_needs_headroom` → mark it as a candidate.
 - If candidates exist → suggest the term/ratio combo that fits, prioritizing the **shortest term that still fits** (less total profit paid).
 - If no candidates → suggest increasing down payment (lower financing ratio) and re-check before suggesting a longer term.
- 6. **Wants > 30% funded by expensive revolving credit card debt**
 - **Classification note (resolves overlap with the needs>50% rule):** this section only applies when the revolving balance traces back to *want* purchases. If the same revolving balance traces back to *need* spending (e.g. covering rent shortfalls), it is not this section's problem — route it to the base logic's "needs > 50%" monthly adjustment instead (temporarily reduce savings / keep wants at 20–30%). A given balance is handled by exactly one of the two paths, never both.
 - **Signal:** wants are over 30% **and** the user is carrying a revolving balance (minimum payments/cash withdrawals), not just using the card as a pass-through.
 - **Logic:**
 - **How `revolving_balance_source` is determined (v1 heuristic):** don't trace individual transactions. Instead, look at wants% in the months the balance was building up:
 - IF wants% was > 30% in most of the months the balance grew:
 - `revolving_balance_source = "wants"`
 - ELIF needs% was > 50% in most of the months the balance grew:
 - `revolving_balance_source = "needs"`
 - ELSE:
 - `revolving_balance_source = "unclear"` → don't fire either section; surface a generic "reduce high-APR card debt" note with no root-cause framing
 - If `wants% > 30%` and `credit_card_revolving_balance > 0` and `revolving_balance_source == "wants"`:
 - If a **lower-APR** card tier is available for the user's profile → suggest migrating the revolving balance to that lower-APR tier (never suggest a higher tier, and never suggest raising the limit).
 - If the "want" behind the balance is a large one-off item that qualifies under a fixed-installment product (e.g., Personal Financing — Cars) → suggest converting it to the installment product (fixed margin ~3–4% vs 38%+ APR is a better structure).

- **Note:** this is structural cost reduction; it doesn't change wants/needs classification.
- 7. **Credit card fee mismatch (small "wants leak")**
 - **Signal:** recurring fees (annual/cash-withdrawal/reissuance) are being paid on a card tier that doesn't match actual usage.
 - **Logic:**
 - If `annual_fee_paid > 0` and usage matches a cheaper tier → suggest downgrading (Classic/Gold/mada; many mada tiers have SAR 0 annual fee).
 - If frequent international transactions and current card is a basic mada tier → suggest a Visa tier with discounted international rate (e.g., 1.15% vs 2.65%/2.2% on mada) and show a breakeven point (annual fee vs transaction savings).
- 8. **Business/self-employed users → irregular income smoothing**
 - **Signal:** user is flagged as business owner/self-employed AND income is irregular enough that monthly 50/30/20 checks are noisy (needs may include business operating costs).
 - **Logic:**
 - If `user.is_business_owner` and `cash_flow_gap_detected`:
 - If business is micro → suggest Micro Enterprise Financing (up to SAR 500,000, ~6–8% margin, up to 60/84 months).
 - If small → suggest Small Enterprise Financing (up to SAR 6,250,000).
 - If medium → suggest Medium Enterprise Financing (up to SAR 18,750,000).
 - Only fire this if the gap is a working-capital/cash-flow issue (don't let business suggestions leak into personal wants overspend nudges).

Suggestion priority order (if multiple fire in the same month):

1. **Free wins first** — idle cash → savings tier.
2. **Cost reduction on existing debt** — high-APR revolving balance → lower-cost structure.
3. **Right-sizing a planned need purchase** — car/real-estate term selection.
4. **Fee optimization** — card tier downgrade/upgrade.
5. **Business financing** — only for flagged business users, isolated from personal ratios.

What this layer never does:

- Never auto-applies for, opens, or moves money into a product.
- Never suggests a product that increases total cost (higher APR, more borrowing) unless it's the only way to keep needs $\leq 50\%$ and the user explicitly asked for financing options.
- Never suggests SME products to a non-business profile.
- Never repeats the same suggestion inside the cooldown window.

